WPT Day 8 Morning

Wed 28 Oct 26 · Request body vs URL parameters, why POST is required, why `request.body` is undefined until configured, Express middleware (`next` , `app.use`), `express.json()` , and what an API really is

---

Table of Contents

1. Request Body: Concept and Motivation
2. GET vs POST: Method Compatibility
3. Why request.body Returns undefined by Default
4. Middleware: Concept and Role
5. Defining and Reusing Middleware
6. Enabling JSON Body Parsing with express.json()
7. APIs: What They Are and Why They Matter
8. Combined Worked Example: Add API with Middleware and JSON Body
9. Quick Revision Checklist
10. Practice Exercises
11. Practice Answers

Request Body: Concept and Motivation

The problem URL parameters cannot solve

So far in the course, data reached the server mainly through **URL parameters** (for example `/sum/5/8` , read with `request.params`). That pattern is fine for small, non-secret values that fit cleanly in a path. It fails for the kinds of payloads real applications send every second.

Without a second channel for data, you would be forced to put passwords, OTPs, banking details, long JSON documents, and arrays into the URL itself. That is insecure, length-limited, and awkward for structured data.

Two ways to pass data to a server (so far)

Channel	How the server reads it	Typical shape
URL parameters	`request.params`	Path segments: `/add/5/8`
Request body (this session)	`request.body` (after middleware)	JSON, form fields, files, etc.

Limitations of URL parameters (lecture list)

1. **Not suitable for sensitive data** — passwords, OTPs, banking details, secrets. URLs are logged in browser history, proxy logs, server access logs, and referrer headers. Secrets in the path leak.
2. **Hard character limit of about 2,000 characters** on the URL. Large payloads simply do not fit.
3. **Cannot send arrays or JSON objects cleanly** via the URL. Nested structures become ugly encoding gymnastics rather than a natural data format.

What a request body is

A **request body** is data a client **attaches to a request** that travels to the server **without appearing in the URL**.

- It can carry **JSON, arrays, form data, files, or any other data type**.
- It is the **reverse direction** of a response body:
- **Response body** = data the **server** sends to the **client**.
- **Request body** = data the **client** sends to the **server**.

Once you see that symmetry, the name "body" stops being mysterious: both directions can carry a payload; only the direction changes.

The contract: keys fixed, values per call

Backend developers **define which keys and value pairs** the request body must contain. Those **keys stay fixed** across all requests; the **values differ** per call.

Example contract for an add endpoint:

```
{ "x": 5, "y": 8 }
```

Every client that wants to use that route must send an object with keys `x` and `y`. Frontend developers receive this contract via documentation and implement accordingly — they do not invent their own key names on a whim.

> **Exam:** Request body = client → server payload that does **not** appear in the URL. Contrast with response body (server → client). List the three URL-parameter weaknesses: secrets, ~2,000-char limit, poor fit for arrays/JSON.

> **Pitfall:** Thinking "body" only means HTML `<body>`. In HTTP, body means the message payload after the headers — for both requests and responses.

GET vs POST: Method Compatibility

The problem: GET cannot carry a body

Even if you understand request bodies, choosing the wrong HTTP method makes the body useless. **A request body is incompatible with GET.** GET requests are not meant to carry a body; if you need to send body data, you need a different method.

Mechanism: POST is required for body data

- **POST** is required whenever a client needs to send data in the **request body**.
- Frontend code sends POST **programmatically** — not by typing a URL into the browser address bar (the address bar is inherently a GET navigation).

Sending POST from the frontend with `fetch`

The second argument to `fetch` is a **config object**:

- `method: "POST"` must be specified **explicitly**. If you omit it, `fetch` **defaults to GET**.
- The `body` key holds the JSON payload to send to the server.

```
fetch("http://localhost:3000/add", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ x: 5, y: 8 }),
});
```

Background: `JSON.stringify` turns a JavaScript object into a JSON **string** for the wire. The matching server middleware (`express.json()`) turns that string back into an object. Without both sides agreeing on JSON, you get strings or `undefined` instead of usable fields.

Axios is an alternative HTTP client for the same job; the lecture notes it will be covered later in the course.

Testing POST without a frontend: Postman

Backend developers use **Postman** (or Thunder Client) to verify server functionality **without building a frontend**:

1. Select **POST** as the method.
2. Enter the target URL.
3. Go to **Body** → **Raw** → **JSON**.
4. Define the JSON payload, for example `{ "x": 5, "y": 8 }`.

Same idea with Thunder Client in the editor: pick POST, paste JSON in the body panel, send.

Tool	Role in this lecture
Browser address bar	GET only — cannot send a request body
`fetch` (frontend)	Programmatic POST with `method` + `body`
Axios	Same idea; covered later
Postman / Thunder Client	Backend testing without a UI

> **Exam:** GET cannot carry a request body; POST is required. `fetch` defaults to GET unless `method: "POST"` is set. Postman path for JSON body: Body → Raw → JSON.

> **Pitfall:** Sending a body on a GET route in Postman and assuming Express will magically expose it the same way as POST. Method and body support are tied together in practice for this course's model.

Why request.body Returns undefined by Default

Live demonstration (lecture)

A POST was sent to `/add` with body `{ "x": 6, "y": 2 }`. The server logged `request.body` and got `undefined`.

Important diagnosis from the lecture: the **client sent the data correctly**. The fault was entirely on the **server not being configured to read it**.

Why Express leaves the body alone

Express **deliberately does not process the request body by default**.

Reason: a request body can contain **many different data types**:

- Plain text
- JSON
- XML
- HTML
- JavaScript code

- Binary / file data

Express **cannot know in advance** which type the developer intends to use, so it **leaves that decision entirely to the developer**.

Until the developer configures body parsing:

- `request.body` will **always** be `undefined`.

Key note from the lecture: Express is **fully capable** of processing all these types; it simply **does not activate** any processing until told to.

Situation	What you see
POST with JSON, no body middleware	`request.body` → `undefined`
Same POST after `express.json()`	`request.body` → parsed object, e.g. `{ x: 6, y: 2 }`
Blame the client first?	No — lecture: client was correct; server was unconfigured

> **Exam:** `request.body` is undefined by default because Express does not auto-parse bodies (many possible types). Capability ≠ default behaviour.

> **Pitfall:** Debugging the Postman JSON for hours when the real bug is missing `express.json()` / `app.use(express.json())` on the server.

Middleware: Concept and Role

The problem: you need work *before* the route handler

A route handler should focus on its job (add two numbers, save a user, return a profile). Cross-cutting concerns — "is this user logged in?", "do they have a subscription?", "parse the JSON body" — belong **between** arrival of the request and the handler. That slot is **middleware**.

Definition

Middleware is a function that executes **between** the arrival of a request and the start of request processing (the route handler).

Without vs with middleware

```
Without middleware:
  request arrives → route handler callback runs immediately

With middleware:
  request arrives → middleware runs first → then (if allowed) route handler runs
```

The three arguments: `request`, `response`, `next`

Middleware has access to:

Argument	Meaning
`request`	Same request object the route handler will see
`response`	Same response object — middleware can send a reply itself
`next`	A reference to the **next** function in the chain (next middleware or the final route handler)

- Calling `next()` passes control **forward** to the next function in the sequence.
- **Not** calling `next()` **halts** the chain entirely.

Live demos from the lecture

1. Middleware logged `"control is in middleware"` but **never called** `next()` → the **client hung indefinitely** waiting for a response.
2. After adding `next()`, the route handler ran and the response arrived successfully.

What middleware can do

- **Allow or block** request processing based on conditions.
- Check whether a user is **authenticated** (logged in).
- Check whether a user has an **active subscription**.
- Verify **payment status** or a specific **subscription tier**.
- **Modify or add properties** on the request object before the handler sees it.
- **Send its own response** directly if a condition fails (e.g. "user is not authenticated") — and then **not** call `next()`.

Chaining multiple middlewares

Multiple middlewares can sit on one route:

```
M1 → M2 → M3 → route handler
```

Example concerns:

- M1 — check login
- M2 — check subscription
- M3 — check subscription tier

Any middleware can **stop** the chain and send its own response. If M1 blocks, **M2 and M3 never run**; the client receives **M1's** response only.

```
Request
   |
   v
[ M1: authenticate ] --fail--> respond 401; STOP
   | ok (next)
   v
[ M2: subscription ] --fail--> respond 403; STOP
   | ok (next)
   v
[ M3: tier check ] --fail--> respond 403; STOP
   | ok (next)
   v
[ Route handler ]
```

Exam: Middleware = function between request arrival and handler; signature `(req, res, next)`; must call `next()` to continue or send a response and stop. Chaining: earlier failure skips later middleware.

Pitfall: Forgetting `next()` and wondering why Postman spins forever. That hang **is** the missing `next()` (or a missing `res` send).

Defining and Reusing Middleware

Pass by reference — do not call it yourself

Middleware is defined as a **named function**, then passed **by reference** into routes — **not** called directly with `()`.

Express calls the middleware automatically at the right time; the developer only supplies the definition.

```
function myMiddleware(req, res, next) {
  console.log("control is in middleware");
  next();
```

```
}

// Correct: pass the reference
app.get("/middleware-demo", myMiddleware, (req, res) => {
  res.send("handler ran");
});

// Wrong: myMiddleware() would invoke it immediately and pass its return value
```

Lecture framing: assigning a named function to a variable / using a named function, then passing `myMiddleware` into `app.get( ... )`, is the pattern. Calling `myMiddleware()` would be wrong.

Per-route vs global registration

Scope	How	Trade-off
A few routes	Pass the reference into each route	Works; duplicates the reference; updating later means touching many places
All routes	`app.use(myMiddleware)` **before** route definitions	Every current and future route runs it first

Live demo pattern from the lecture:

1. Remove `myMiddleware` from individual routes.
2. Add `app.use(myMiddleware)` near the top.
3. Requests to `/middleware-demo` **and** the base URL both triggered the middleware **without** any per-route mention.

Naming convention

Give each middleware a **descriptive name** for its concern:

- `authenticateUser` — login checks
- `checkSubscription` — subscription validation

Order matters

Sequence of middleware is not cosmetic:

1. **Authenticate** the user **before** checking their subscription.
2. If the user is not authenticated, there is **no user data** to check a subscription against.

Escape hatch (preview only)

When **most** routes need a middleware but **one** does not, the lecture points to Express's **router** object to group routes — covered later. Do not invent a complex skip system yet; know that routers are the intended tool.

> **Exam:** Pass middleware by reference (`myMiddleware`), not `myMiddleware()` . Global: `app.use( ... )` before routes. Order: auth before subscription-dependent checks.

> **Pitfall:** Writing `app.use(myMiddleware())` with parentheses — that runs the function at registration time and usually registers `undefined` as middleware.

Enabling JSON Body Parsing with express.json()

Factory, not middleware itself

`express.json()` is a **factory function**, not a middleware by itself.

- Calling it **returns** a middleware function that parses **JSON** data from the request body.
- Express docs phrasing (as cited in class): it **returns the middleware that only parses JSON**.

```
const jsonParser = express.json(); // jsonParser is the actual middleware
```

Per-route vs global (recommended)

On a single route — pass `express.json()` inline before the handler:

```
app.post("/add", express.json(), (req, res) => {
  // req.body available here
});
```

Globally (recommended for real-world apps) — register once before routes:

```
app.use(express.json());
```

Every route can then read JSON from the request body without repeating `express.json()` per route. **GET routes are unaffected** in practice for this course's model, since GET is incompatible with request body anyway.

Live `/add` demonstration

Stage	Log of `request.body`
Before `express.json()`	`undefined`
After adding `express.json()`	`{ x: 8, y: 2 }` (full parsed object)

Accessing values:

```
// Direct property access
request.body.x;
request.body.y;

// Destructuring
const { x, y } = request.body;
const z = x + y;
res.json({ sum: z });
```

Lecture result: with `x: 5`, `y: 2` → response `{ sum: 7 }`.

Types are preserved (advantage over URL params)

Through the request body (JSON):

- A **number** sent by the client arrives as a **number** on the server.
- A **boolean** sent by the client arrives as a **boolean**.

With URL parameters, **every** value arrives as a **string** and must be parsed manually (`parseInt`, etc.). That contrast is a major reason to prefer JSON bodies for structured input.

Source	`"5" / 5` on the wire	Type on server
URL param	path segment	always string
JSON body (after `express.json()`)	JSON number / boolean / etc.	preserved

> **Exam:** `express.json()` returns JSON-parsing middleware. Prefer `app.use(express.json())`. JSON body preserves number/boolean types; URL params are always strings.

> **Pitfall:** Calling `express.json` without `()` — you must **invoke** the factory so Express receives the middleware function it returns.

APIs: What They Are and Why They Matter

Interface first (ATM and camera analogies)

API stands for **Application Programming Interface**.

An **interface** is the middle layer that lets one party use the functionality of another:

Real-world interface	What it sits between
ATM screen / keypad	Person ↔ ATM hardware
Camera app	User ↔ camera hardware

You do not rewire the ATM; you use its interface. You do not solder the camera sensor; you open the camera app.

Definition

An **API** is the interface that lets **one application use the functionality of another application programmatically**.

The routes built throughout this session **are** APIs — created even before the name was emphasised.

Indigo flight-booking example

Indigo exposes APIs such as:

- Search flight
- Book ticket
- Cancel ticket
- Update booking

Who consumes those **same** APIs?

- Indigo's own website
- Indigo's mobile app
- MakeMyTrip, GoIbibo, Yatra, …

Indigo did **not** give third parties access to its entire database. It gave them **specific API URLs**. Each caller sends a **JSON request body** and receives a **JSON response** — that is the entire contract.

PNR status example

Hundreds of websites can show PNR status by calling the **railway's API**, not by accessing the railway database directly.

Benefits of building a backend as APIs

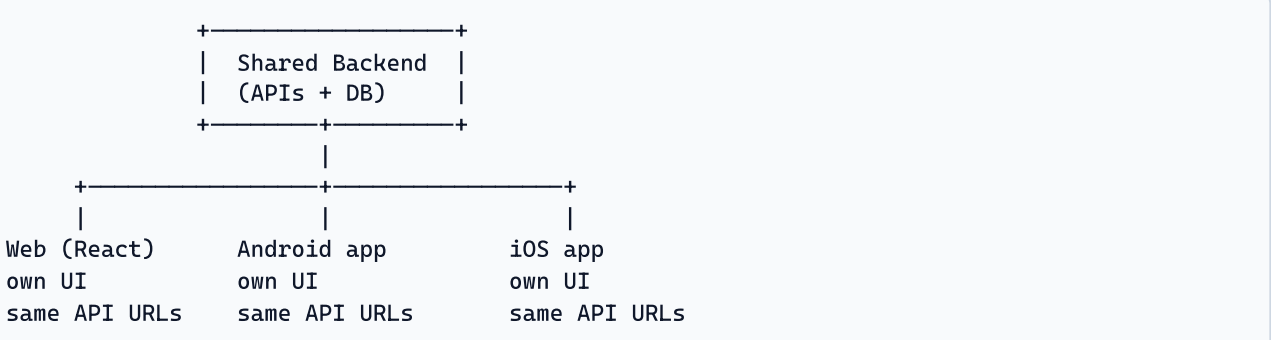
1. **One backend** serves a website, an Android app, and an iOS app simultaneously.
2. Frontend and backend are **fully decoupled** and can be developed **in parallel** by separate teams.
3. Backend developers focus on routes, request/response bodies, database queries, and business logic.
4. Frontend developers focus on UI; they only need the **API URL**, expected **request body** format, and **response** format.
5. The URL of an API reveals **nothing** about the backend technology used to build it.
6. Switching backend technology (e.g. Node.js → Java) requires **zero** changes on the frontend (same URLs and JSON contracts).
7. Switching frontend framework (e.g. React → Angular) requires **zero** changes on the backend.

Multi-platform social app sketch

Single backend with APIs for login, logout, signup, get profile, get posts, save comment, post photo, etc.:

- Website frontend (React) calls those APIs.
- Android app has its own UI but calls the **exact same** APIs.
- iOS app has its own UI (different native components → different look) but calls the **exact same** APIs.
- No need to rebuild the backend or database for each platform.

At scale, the same pattern: YouTube, Flipkart, Amazon, WhatsApp. WhatsApp Android and iOS **look** different (platform-native UI) but share the **same backend and data** — same messages appear on both because both call the same APIs.



> **Exam:** API = programmatic interface between applications. One backend, many clients; JSON request/response contract; no direct DB access for third parties (Indigo / PNR examples). Decoupling → swap frontend or backend independently.

> **Pitfall:** Defining API only as "a URL." A URL alone is not the contract — method, request body shape, and response shape complete it.

Combined Worked Example: Add API with Middleware and JSON Body

One runnable Express file that ties together: global `express.json()`, a logging middleware with `next()`, a POST `/add` that reads the request body, and a second route that demonstrates middleware chaining order. Save as `day8-add-api.js`, run with `node day8-add-api.js`, then test with Postman.

```
const express = require("express");
const app = express();
const PORT = 3000;

// 1) Global JSON body parser (factory → middleware)
app.use(express.json());

// 2) Global logger middleware — must call next()
function logRequest(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next();
}
app.use(logRequest);

// 3) Optional gate middleware used only on /add
function requireNumbers(req, res, next) {
  const { x, y } = req.body || {};
  if (typeof x !== "number" || typeof y !== "number") {
    return res.status(400).json({
      error: "Body must be JSON with numeric x and y",
    });
    // no next() — chain stops here
  }
  next();
}

// POST /add — body: { "x": 5, "y": 2 } → { "sum": 7 }
```

```
app.post("/add", requireNumbers, (req, res) => {
  const { x, y } = req.body;
  const z = x + y;
  res.json({ sum: z });
});

// GET demo — no body; middleware still logs via app.use
app.get("/middleware-demo", (req, res) => {
  res.send("handler ran after logRequest");
});

app.listen(PORT, () => {
  console.log(`Server on http://localhost:${PORT}`);
});
```

End-to-end trace (Postman)

1. Method **POST**, URL `http://localhost:3000/add`.
2. Body → Raw → JSON: `{ "x": 5, "y": 2 }`.
3. Server path: `express.json()` fills `req.body` → `logRequest` logs and calls `next()` → `requireNumbers` sees numbers and calls `next()` → handler computes `sum: 7` → JSON response.
4. If body is `{ "x": "5", "y": 2 }` (string `x`), `requireNumbers` responds **400** and the handler never runs — same "early middleware can stop the chain" idea as login/subscription checks in the lecture.
5. If you comment out `app.use(express.json())` and resend, `req.body` is `undefined` and the gate fails — matching the live undefined demo.

Assignments from the lecture (same body pattern)

Use the same POST + JSON body approach for practice before the evening session:

Route	Request body	Response idea
Factorial	`{ "number": n }`	factorial of `n`
`/calculate-bill`	`{ "rate", "quantity" }`	total, discount amount, net payable
`/circle`	`{ "radius" }`	area and circumference

Discount bands for `/calculate-bill` (on **total** = rate × quantity):

Total amount	Discount
3,000–7,000	7%
7,001–15,000	10%
Above 15,000	15%

Evening preview (not implemented here): real-time APIs with live database connectivity — save and retrieve records from a real DB.

Quick Revision Checklist

Topic	One-line takeaway
Request body	Client→server payload off the URL; OK for JSON/arrays/files; not for secrets-in-URL
URL params limits	Secrets leak; ~2,000-char cap; arrays/JSON awkward
GET vs POST	GET cannot carry a body; POST required; `fetch` defaults to GET
Postman	POST → Body → Raw → JSON to test without a frontend
`request.body` undefined	Express does not parse bodies until you configure it
Middleware	Runs between arrival and handler; `(req, res, next)`
`next()`	Must call to continue; omit + no response → client hangs
Middleware chain	M1→M2→M3→handler; early stop skips the rest
Register middleware	Pass by reference; `app.use` for global
`express.json()`	Factory returning JSON parser; prefer global `app.use`
Types in JSON body	Numbers/booleans preserved (unlike URL string params)
API	Programmatic app-to-app interface; one backend, many clients

---

Practice Exercises

Concept

1. Why are passwords and OTPs a bad fit for URL parameters? Give two concrete leak paths.
2. In one sentence each, define request body and response body, and state how they differ in direction.
3. Why does Express leave `request.body` undefined until you opt in to parsing?
4. What are the three arguments to a middleware function, and what does `next` hold?
5. Using the ATM analogy from the lecture, explain what an API is without naming a programming language.

Application

6. Write the Postman steps to POST `{ "x": 5, "y": 8 }` to `http://localhost:3000/add`.
7. Write an Express snippet that registers JSON parsing globally and a POST `/add` that returns `{ sum }` using destructuring from `request.body`.
8. Write a named middleware `authenticateUser` that logs a message and calls `next()`, and attach it with `app.use` so every route runs it.
9. For `/calculate-bill`, given `rate: 100` and `quantity: 80`, compute total, discount amount, and net payable using the lecture's discount bands.

Predict output

10. Middleware logs `"in MW"` and does **not** call `next()`. The route handler would send `"ok"`. What does the client observe?
11. `app.post("/add", (req, res) ⇒ res.json(req.body))` with **no** `express.json()`. Postman sends raw JSON `{ "x": 1 }`. What is the response body likely to show for `req.body`?
12. Client JSON has `"x": 5` (number) via body vs URL param `/add/5`. After correct parsing setup, what **types** does the server see for `x` in each case?

Spot the bug

13. `app.get("/add", express.json(), (req, res) ⇒ { res.json(req.body); });` with Postman GET and a JSON body. What conceptual mistake matches the lecture?
14. `app.use(express.json);` — what is wrong?
15. `app.post("/add", myMiddleware(), handler)` — what is wrong with `myMiddleware()`?

Practice Answers

1. Sensitive values in URLs appear in browser history, server/proxy logs, and often referrer headers — so passwords/OTPs can leak. URLs are also shared and bookmarked. (Any two of those leak paths suffice.)

2. Request body: data the **client** attaches and sends to the **server** (not in the URL). Response body: data the **server** sends back to the **client**. Direction reverses; both are payloads.

3. Bodies can be many types (text, JSON, XML, HTML, JS, binary/files). Express will not guess; until you configure a parser, `request.body` stays `undefined`. Express *can* parse them — it just does not by default.

4. `request`, `response`, and `next`. `next` holds a reference to the next function in the chain (next middleware or the route handler).

5. An interface sits between you and a machine's functionality (ATM UI ↔ ATM hardware). An API is that idea for programs: one application uses another's functionality through a defined programmatic interface (URLs + JSON contracts), not by touching the other app's internals/database.

6. Open Postman → method POST → URL `http://localhost:3000/add` → Body → Raw → JSON → paste `{ "x": 5, "y": 8 }` → Send.

7.

```
const express = require("express");
const app = express();
app.use(express.json());
app.post("/add", (req, res) => {
  const { x, y } = req.body;
  res.json({ sum: x + y });
});
app.listen(3000);
```

8.

```
function authenticateUser(req, res, next) {
  console.log("authenticateUser running");
  next();
}
app.use(authenticateUser);
// define routes after this
```

9. Total = 100 × 80 = **8,000**. Band 7,001–15,000 → **10%** discount. Discount amount = 800. Net payable = **7,200**.

10. The client waits indefinitely (hangs) — no response is sent and the handler never runs.

11. `undefined` (or a JSON response that serialises that), because no body-parsing middleware populated `req.body`.

12. JSON body: `x` is a **number**. URL parameter: `x` is a **string** (`"5"`).

13. Request body is incompatible with GET; POST is required to send body data the way this session teaches. Use POST (and still need `express.json()`).

14. Missing `()` — `express.json` is a factory; you must call `express.json()` so the returned middleware is registered.

15. Parentheses **call** the function immediately. Pass the reference `myMiddleware` so Express can call it later with `(req, res, next)`.